

- Support vector machines (SVMs)
- Decision trees and random forests
- Neural nets (also called NNs)

Unsupervised learning: Here, the training data is unlabelled. It is similar to a system trying to learn without a teacher. In unsupervised learning, the training of the machine or system is done using the information that is neither classified nor labelled. Because of this, the algorithm itself will act on information without any guidance. Here, the machine with this unsorted information tries to recognise similarities, patterns and differences, without any training.

Some of the most important and extensively used unsupervised algorithms are listed below.

Clustering: These classify customers or buyers based on their purchasing patterns:

- K-Means
- Hierarchical cluster analysis (HCA)
- Expectation maximisation

Visualisation and dimensionality reduction:

- Principal component analysis
- Kernel PCA
- Locally-linear embedding (LLE)
- T-distributed Stochastic neighbour embedding (t-SNE)

Association rule learning: These set rules based on certain recognisable patterns of behaviour, like the customer who buys product X may also buy product Y.

- Apriori
- Eclat

Semi-supervised learning: Some algorithms can deal with partially labelled data and much unlabelled data; or sometimes, with a little bit of labelled data. These kinds of algorithms are called semi-supervised learning algorithms.

A photo hosting service like Google Photos is a good example of semi-supervised learning.

Reinforcement learning: RL or reinforcement learning is an area of ML inspired mainly by behavioural psychology. It is totally different from supervised learning. In this algorithm, an agent learns how to behave in an environment by performing actions and seeing the results.

Examples include Google's DeepMind and AlphaGo.

Batch and online learning: Online and batch processing are slightly different, although both perform well for parabolic performance surfaces.

One major difference between them is that the batch algorithm keeps the system weights constant while computing the error associated with each sample in the input. Since the online version is constantly updating its weights, its error calculation (and thus gradient estimation) uses different weights for each input sample. This means that the two algorithms visit different sets of points during adaptation. However, they both converge to the same minimum.

Scikit-learn

In this article, we will primarily explore the Scikit library for

machine learning purposes. Scikit is open source and released under the BSD licence. Most parts of the Scikit library are written in Python and some of its core algorithms are written in Cython (for efficiency and performance).

Scikit-learn's main purpose is to build models; so this library is not well suited for other activities like reading, manipulation of data or the summarisation of data.

The benefits of Scikit-learn are:

- It is one of the most consistent interfaces available today to build ML models. It provides the user with many tuning parameters.
 - Many of the good functionalities are in-built in this library.
 - It also has good documentation that is easy to understand, and has active community support.
 - It covers most of the well-known machine learning models
- However, it does have some drawbacks:
- Many geeks mention that in the beginning, Scikit-learn is somewhat harder to learn, compared to R.
 - There is less emphasis on model interpretability.

Scikit-learn is a machine learning library for Python. It features several regression, classification and clustering algorithms including SVMs, gradient boosting, k-means, random forests, etc. I have given a brief overview of these algorithms earlier, keeping this in mind. This library is also designed to work with other Python based libraries like that of NumPy and SciPy.

A combination of these three libraries is well suited for many machine learning models.

Installation

The installation process of Scikit is easy, and it is assumed that the latest version of the NumPy (1.8.2 or above) and SciPy (0.13.3 or above) library packages are already preinstalled and supported in your systems. Once these prior dependent packages are installed, the installation of the Scikit library can proceed.

 **Note:** It is assumed that Python 3.3 or a later version is used for these models.

Depending on the framework or Python environment in use, Scikit can be installed with the following set of commands. For Pip installation, give the following command:

```
ubuntu@ubuntu:~$ pip install scikit-learn
```

If the Anaconda package is in use, then use the following command (this step varies from the Conda prompt based on Windows, Mac or Linux platforms). Use the IPython terminal (now called Jupyter):

```
# conda install scikit-learn
```

Once the installation is done, the Scikit library can be used to build and predict models in Jupyter or